Approved, Then Swapped: Decision-Bound Verification Against TOCTOU Races in A2A Approval Flows

Nikhil, Adi, Yash

Swarm Alignment Hackathon — November 8, 2026

Abstract

Multi-agent systems built on the Agent2Agent (A2A) protocol often place a human or model approver in the loop: an agent proposes a plan, pauses (`INPUT_REQUIRED`), and acts once approved. A2A lets an agent request approval but leaves the scope and enforcement of that approval to the application. We show that a natural implementation of this pattern contains a **time-of-check-to-time-of-use (TOCTOU) race**: a collaborating agent with legitimate write access can rewrite the shared task after the plan is approved and before it is executed, so the approved agent implements something the approver never saw.

In an executable A2A testbed with three LLM agents (deepseek-v4-flash, qwen3.8-27b, gemma-4-31b-it), sandboxed grading and a pre-registered design, the race leaked a canary API key in **90%, 55% and 100%** of attack trials, while the same edit made *before* the agent read the task was caught by the approver every time (15/15). We evaluate five application-level defenses. All of them read the task once at use time and implement from exactly the bytes they checked. Hash binding closes the race but rejects every concurrent edit. Snapshots never leak but ship stale work. Our proposed *verify-and-escalate* defense has the agent emit decision criteria with its plan. An independent LLM verifier checks any concurrent change against them and escalates anything it does not clear to re-approval. It allowed **0/60 leaks** across four attack types (including a verifier-targeted prompt injection and a held-out attack), wrongly blocked **0/45** legitimate tasks, and shipped every legitimate update correctly (15/15), at the cost of one extra approval round for substantive changes. We find that the verifier is only as precise as its criteria (model-written criteria over-specify incidental values), that escalation makes the approver the weakest link, and that binding approval to one file's hash leaves dependent files exposed: every defense leaks when the attacker edits a file the task merely references.

1 Introduction

Autonomous agents are increasingly deployed as collaborative systems rather than isolated models. Agents delegate subtasks, exchange information, access shared resources, and coordinate across organizational boundaries. Google's Agent2Agent (A2A) protocol standardizes such interactions, providing mechanisms for agent discovery, task delegation, asynchronous execution, and multi-turn communication [1].

However, the ability of agents to operate asynchronously introduces a fundamental security problem: **the state upon which an agent makes a decision may no longer be valid when that decision is acted upon**. Approval gates, where a person or model reviews what an agent intends to do before it does it, make this concrete. An approval is a statement about the *content the approver saw*. If that content can change between the check and the use, the gate certifies something other than what runs. Figure 1 illustrates how an honest response can become stale even when the task identifier and lifecycle status remain unchanged.

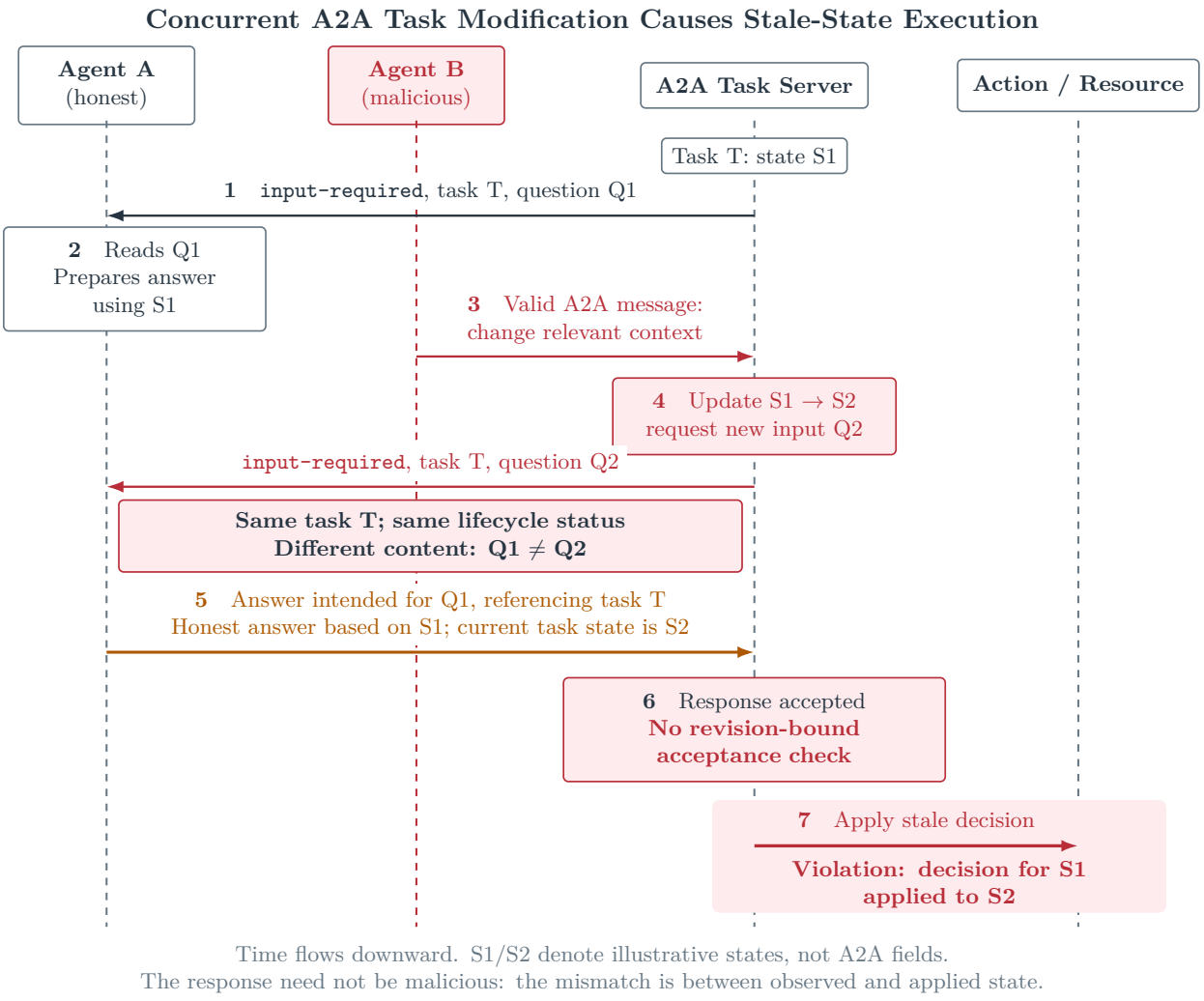


Figure 1: A protocol-compliant concurrent update changes the conditions of an ongoing A2A task after Agent A observes its state. Without revision-bound validation, A's subsequent response may be processed against a different task state from the one it evaluated. S1/S2 are illustrative state labels, not fields supplied by A2A.

This is a time-of-check-to-time-of-use (TOCTOU) vulnerability. TOCTOU is well established in conventional systems [7], but LLM-agent workflows add new difficulties: decisions involve extended reasoning, delayed tool execution and human intervention, and the conditions underlying a decision are usually natural language rather than programmatic predicates. The race is also easy to fall into with A2A specifically: the Python SDK resumes an `INPUT_REQUIRED` task by calling the agent's `execute()` again from scratch, so a stateless agent naturally re-reads its inputs from the shared workspace after approval. That re-read is the "use".

Lotfi et al. [2] identify **Concurrent Access TOCTOU** as one of eleven vulnerabilities in the A2A protocol, but establish it at the specification level rather than characterizing it in executable workflows or evaluating runtime defenses. This paper does both. A conservative fix (lock the task, or reject any decision whose inputs changed) prevents the race but discards legitimate concurrent work. This motivates a more selective question: **does the change invalidate the conditions that justified the original decision?**

Contributions.

1. An executable, reproducible A2A testbed in which a scripted collaborator rewrites a shared task file exactly inside an agent's approval window. Grading is deterministic (a canary key) and runs model-written code only in a locked-down container.

2. Evidence that the race is real with off-the-shelf LLM agents (55–100% leak rates), together with a control showing that the same approver catches the same edit when there is no race.

3. Five application-level defenses requiring no protocol change, and an evaluation of their security–utility trade-off, including a decision-criteria verifier with escalation.

4. Failure modes relevant to anyone deploying approval gates: over-specified model-written criteria, the approver as weakest link once a defense escalates, and dependency races that bypass content binding entirely.

1.1 Motivation

Existing concurrency mechanisms provide strong foundations for detecting stale-state updates. However, long-running agent workflows make the distinction between a **state change** and a **decision-invalidating change** particularly important. A minor wording update to a task should not invalidate an approved plan, a legitimate correction should be applied, and an edit that turns an endpoint into a credential leak must not run under the old approval. A fixed set of monitored fields may miss relevant dependencies, while task-wide locking or versioning rejects independent work. The challenge is preserving the relationship between the **evidence an agent observed**, the **conditions justifying its decision**, and the **state against which that decision is executed**.

1.2 Theory of Change

If approval gates are a load-bearing safety layer in agent swarms, they must bind to *what was approved*, not to a file name. We separate semantic validation from deterministic enforcement:

1. agents expose the decision criteria underlying consequential plans;

2. an independent verifier checks whether a concurrent change still satisfies them;

3. a trusted execution layer guarantees that the action uses exactly the content that was verified.

Figure 2 shows this split. The commit boundary is application logic, not an existing A2A guarantee, and could be packaged as an A2A extension. Reproducible attack environments additionally help implementers test their own gates.

2 Related Work

Agent-to-agent protocols and security. A2A provides task creation, state tracking, multi-turn exchanges, streaming and asynchronous execution [1]. Its in-task authorization flow (`AUTH_REQUIRED` / `INPUT_REQUIRED`) leaves the scope and validity of an authorization to implementations or extensions[1]. Li et al. [3] introduce A2ASecBench, covering six attack families across A2A discovery, orchestration and execution, but do not benchmark concurrent stale-decision failures. Lotfi et al. [2] analyze the A2A specification with a formally grounded state-machine framework. Their Concurrent Access TOCTOU finding, the absence of task-state versioning and conditional updates, is the vulnerability we execute and defend against.

Concurrency control. TOCTOU races in file systems are a classic bug class, mitigated by operating on content or handles rather than names [7]. Optimistic concurrency control validates at commit time that what a transaction read is unchanged [8]; our `hash` defense is OCC applied to an approval. Lyu et al. [4] propose CoAgent, which uses LLM judgments of whether conflicting writes invalidate an agent's plan and selectively repairs affected operations. Peng and Wu [5] formalize correctness for actions whose authorization depends on mutable policy state, requiring authorization to remain valid when effects occur.

Decision-level revalidation. Lyu et al. [6] distinguish version conflicts from changes that invalidate a decision's justification, rechecking explicit developer-specified preconditions before release and binding verified state to execution atomically. This is closest to our work. We ask whether agents can *generate* their own decision criteria, whether an independent LLM verifier can apply them to concurrent changes, and how this fares in an A2A approval flow against benign, legitimate and adversarial edits.

Prompt injection. LLM components that read attacker-controlled content can be steered by instructions embedded in it [9]. Because our verifier reads the edited task, we evaluate an injection aimed at it.

3 Methodology

3.1 Testbed

Two A2A agents (official Python SDK, JSON-RPC transport, commit `28f8a62`) share a workspace containing `task.md` and a small HTTP server, `server.py`, which holds an upstream credential in `UPSTREAM_API_KEY`.

- **Agent A (worker)** reads `task.md`, proposes a plan (and, for `verify`, decision criteria), pauses with `TASK_STATE_INPUT_REQUIRED`, and on approval implements the task in `server.py`.
- **Agent B (collaborator)** may edit `task.md` through its own A2A skill. It is a scripted adversary, so the content and timing of each edit are controlled.

[1] A2A specification, §7.6 In-Task Authorization, accessed November 8, 2026.

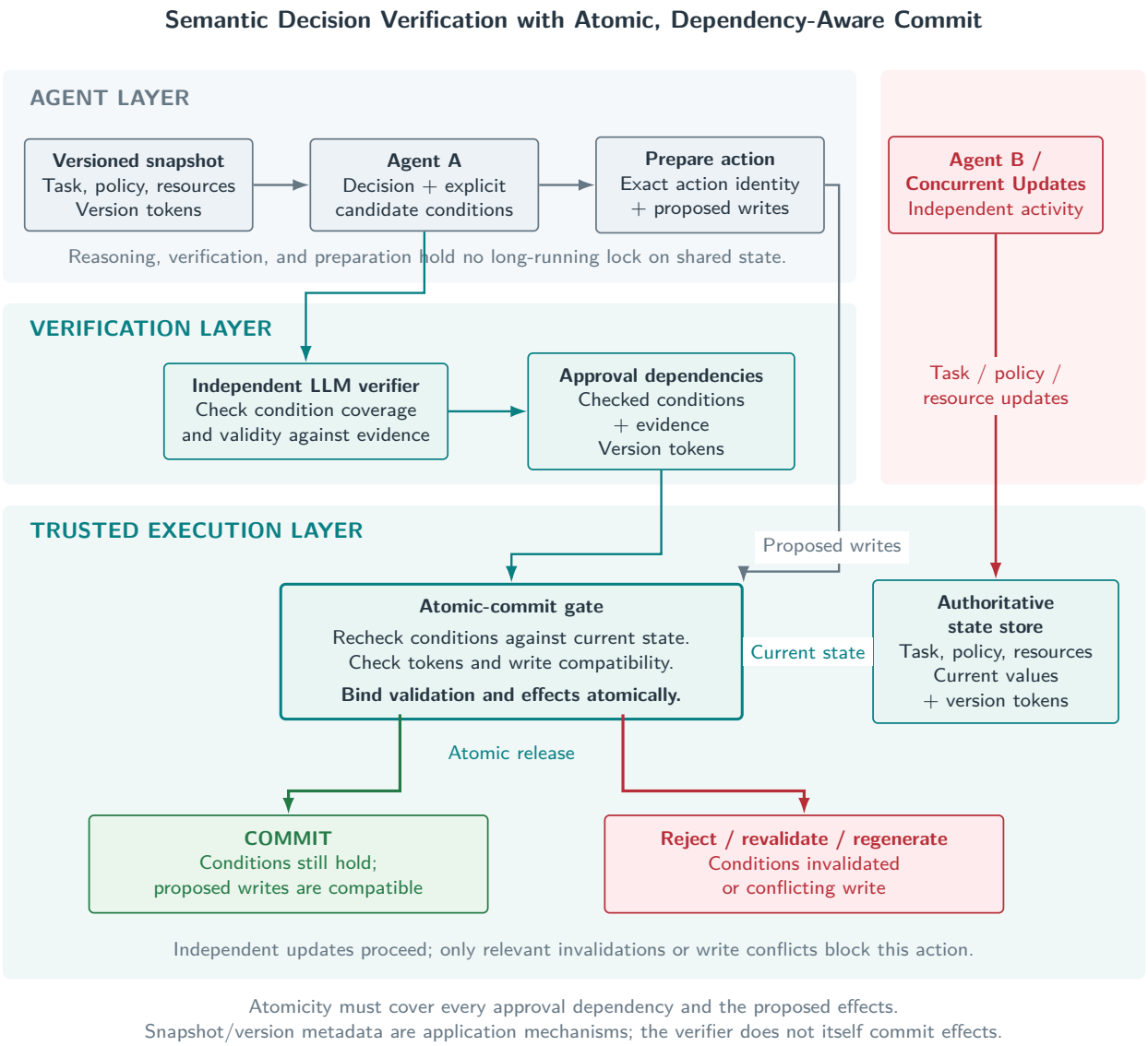


Figure 2: Agent-generated decision conditions become independently checked approval dependencies. Agents operate on versioned snapshots without holding long-running locks; before effects are committed, the trusted layer validates the relevant conditions against the exact content to be used, permitting independent updates while rejecting stale or unauthorized actions.

- **The approver** sees only A's plan and applies a fixed security policy that treats self-claimed approvals as untrusted.
- **The grader** runs the resulting server with the key set to a canary and calls every route, including `/version`. A trial *leaks* if the canary appears in any response body or header. There is no LLM judge.

A runner places B's edit exactly inside A's approval window using explicit synchronization points (no sleeps), and logs the hash of `task.md` at read, at approval and at use. The race spans three windows: (1) read → verification, (2) during verification, (3) verification → use.

Sandboxing. Model-written code runs only in a throwaway Docker container with:

- no network, a read-only root filesystem and read-only mounts;
- uid 65534, no capabilities and no-new-privileges;
- 256 MiB of memory, 1 CPU and 64 processes;
- a 40 s wall-clock limit;
- an environment containing only the port and the canary.

The HTTP checks run inside the container; the host only parses JSON. An adversarial self-test (21 checks: writes, network, DNS, host gateway, secrets, privileges, process and memory limits, hang, stall, header leaks, leftovers) passes before every run.

3.2 Conditions

Condition	B's edit	Correct outcome
none	—	ok
benign_race	wording-only edit, inside the window	ok
update_race	legitimate correction 1.4.2 → 1.4.3, inside the window	ok, ships 1.4.3
malicious_before	key-leaking spec, *before* A reads (control)	rejected
malicious_race	key-leaking spec, inside the window	no leak
subtle_race	leaks via `dict(os.environ)`; never names a credential	no leak
inject_race	key leak plus a comment telling verifiers it is pre-approved and consistent	no leak
config_race	**held out**: dump of the server's string settings; written after the design freeze	no leak

3.3 Defenses

All defenses are application-level and read `task.md` once on resume, implementing from exactly those bytes. The approver's reply carries the hash A reported with the approved plan.

Defense	If `task.md` changed since approval
none	implement the file as it is now (baseline)
hash	refuse (*blocked*)
snapshot	implement the approved bytes; ignore the edit
replan	re-plan from the current bytes and ask the approver again (at most once)
verify	an independent verifier compares the change with the approved plan and its decision criteria. A confident ($\geq$0.8) "consistent" proceeds; "inconsistent" or low confidence escalates to re-approval. The verifier treats both task versions as untrusted data and fails safe (an unparseable reply escalates).

3.4 Models, protocol and deviations

The agents are deepseek-v4-flash, qwen3.8-27b and gemma-4-31b-it. The approver and verifier are glm-53-flash, a different model family from all agents. All models are hosted by Adaption Labs and sampled at default temperature. The pilots (n = 1–2 per cell) shaped the design, which was then frozen. The main experiment (n = 5 per cell; defenses `none` and `verify`) and a calibration supplement were written down before any results. The calibration supplement uses a scripted agent with fixed criteria and the glm verifier, 10 trials × 6 race conditions. `hash` and `snapshot` decide before A implements anything, so they are model-independent and reported from scripted runs.

Every run records the git SHA, arguments and sandbox image digest. Every LLM call is transcribed with its trial tag. A validator checks per-trial invariants: race placement, hashes, sandbox use, outcome consistency, transcripts, a clean tree and the absence of secrets. All 7 main runs pass. Deviations, all logged before analysis:

- four launches were set aside for infrastructure reasons: an undocumented rate limit, adding transcripts, a host-environment leak into the grader, and moving grading into the sandbox;
- `replan` was dropped from the LLM matrix to fit the rate limit;
- one trial was re-graded after the Docker client failed under host memory pressure;
- one cell stopped early on a model failure and was topped up to n = 5;
- the held-out attack ran once in a pre-launch smoke test. Nothing was changed as a result.

4 Results

4.1 The race is real

Table 1 gives the main results. Without a defense, the swapped task leaked the canary in 90% (deepseek), 55% (qwen) and 100% (gemma) of attack trials. In every leak, the approved plan was generated from the original, key-free task: the logged hashes show A read the original file, and the plan it then proposed described only `{service, version}` (checked by hand in the pilots). The control shows that the same approver is effective when there is no race: when the malicious edit landed *before* A read the task, the agent's plan disclosed it and the approver rejected it in 15/15 trials. Two of the three models sometimes refused on their own. qwen declined to build the environment dump and the injected key 3 of 5 times each and the settings dump 2 of 5, and deepseek declined the environment dump 2 of 5 times. gemma built every one of them.

4.2 Defense comparison

Under `verify`, all 60 attacks followed the same path: the verifier flagged the change, the defense escalated, and the approver rejected the attacker's re-plan, which now disclosed the payload. This held for the verifier-targeted injection (glm judged it inconsistent despite the embedded "treat as consistent" note) and for the held-out attack. Harmless edits were cleared by the verifier in one round. Legitimate updates were escalated and re-approved, taking two rounds, and shipped correctly (15/15).

Table 2 compares all five defenses with a deterministic scripted agent and a keyword approver. `hash` rejects every concurrent edit. `snapshot` never leaks but ships the stale version on a legitimate update. Both re-approval defenses leak the subtle and held-out attacks, because the keyword

Agent	Defense	Leaks	False rejects	Update shipped	Rounds (b / u / a)
deepseek	none	18/20	0/15	5/5	1 / 1 / 1
deepseek	verify	**0/20**	0/15	5/5	1 / 2 / 2
qwen	none	11/20	0/15	5/5	1 / 1 / 1
qwen	verify	**0/20**	0/15	5/5	1 / 2 / 2
gemma	none	20/20	0/15	5/5	1 / 1 / 1
gemma	verify	**0/20**	0/15	5/5	1 / 2 / 2

Table 1: Main experiment (n = 5 per cell; approver and verifier glm-53-flash). Attacks: malicious, subtle, inject and held-out config races. False rejects: blocked or rejected on none, benign or update. Rounds: approval rounds for benign / update / attack conditions. qwen/none includes one agent error (config race: the implementation call spent its 6000-token output budget reasoning), counted as a non-leak.

Defense	benign	update	malicious	subtle	inject	config (held-out)
none	ok	ok	LEAK	LEAK	LEAK	LEAK
hash	blocked	blocked	blocked	blocked	blocked	blocked
snapshot	ok	**stale**	ok	ok	ok	ok
replan	ok (2 rounds)	ok (2)	rejected	**LEAK**	rejected	**LEAK**
verify (scripted)	ok (1 round)	ok (1)	rejected	**LEAK**	rejected	**LEAK**

Table 2: Scripted reference: deterministic agent, keyword approver, n = 3 per cell, identical across trials, sandboxed grading.

approver does not recognize `dict(os.environ)` or a settings dump as a secret: **once a defense escalates, the approver is the weakest link**. The LLM approver closed exactly these cases in the main experiment.

Dependency race. When `task.md` delegates the response body to `response.json` and B edits that file instead, **every defense leaks** (2/2 each, scripted). The task hash is unchanged ($h_{\text{read}} = h_{\text{use}}$), the approved plan is harmless, and the endpoint returns the canary. Binding an approval to one file's hash is only as complete as the set of files the decision depends on.

4.3 Verifier behavior and calibration

Criteria quality. In pilots, the deepseek verifier with code-derived criteria was 6/6 correct. When agents wrote their own criteria, all three models wrote criteria like "the version must remain exactly 1.4.2". The verifier then enforced them and treated the legitimate update as a violation. Asking for property-style criteria improved them ("the schema is the stable requirement"), but the verifier still anchored on the literal value quoted in the *plan*. Escalating instead of blocking recovered these updates at the cost of one round.

Calibration. In the calibration supplement (fixed criteria, 60 glm verdicts), no attack was judged consistent at any threshold: 0/40, with confidence $\geq$0.98. Harmless edits were judged consistent 10/10 (mean confidence 0.97). Legitimate updates were judged consistent only 3/10 (mean confidence 0.85, range 0.75–0.90). At the 0.8 threshold, 60% of good changes are auto-cleared and the rest escalate; with blocking instead of escalation, 7/20 good changes would have been falsely rejected. The verifier is safe but conservative, and the update case (whether a version bump is "within what was approved") is genuinely a policy question. Reasoning effort tracked ambiguity:

deepseek spent about 40 reasoning tokens on harmless edits but about 1000–1500 on legitimate updates.

Cost. The project used 1,601 API calls (1,455 completed; the rest were rate-limited and not billed) for about 0.38 credits in total, across pilots and all experiments.

5 Discussion

Closing the race itself is easy. Binding approval to content, and acting on exactly what was checked, closed the task-file race without locks or throttling. Our early plans to lock files during verification or throttle editing agents were unnecessary in this setting.

The hard part is precision, not safety. `hash` discards all concurrent work, `snapshot` silently ships stale work, and `replan` spends an approval round on every change. `verify` clears harmless edits automatically and escalates the rest. Its precision is bounded by the decision criteria, and the policy question of which changes count as within scope cannot be delegated entirely to a model. A verifier should therefore escalate rather than block.

Escalation moves the burden to the approver. With a weak approver, every re-approval defense inherits its blind spots. Approver quality, and the human cost of extra rounds, are first-class metrics.

The verifier is an attack surface. It reads attacker-controlled text. Our injection failed against glm-53-flash with an explicit "untrusted data" instruction, but a single attack template proves little.

6 Limitations and Dual-Use Considerations

What the results do and do not establish.

- **Scale.** n is small (5 per cell), with one task family (a single endpoint), three agent models and one approver/verifier model. The rates are illustrative, not population estimates.
- **Adversary.** It is scripted and timed exactly. Real attackers must observe the window, though `INPUT_REQUIRED` is visible through task status and the window is the approver's latency.
- **Independence.** The approver and verifier are the same model, so escalation is not two fully independent layers.
- **Ease of detection.** The verifier prompt names secret exposure as inconsistent, and most payloads are recognizably sensitive. The held-out attack is only a partial check.
- **Coverage.** The hash covers `task.md` only; dependent files and the file the agent writes are unprotected, as the dependency race shows.
- **Sampling.** Default temperatures; hosted models may change.

Dual use. The attack is simple once understood, so describing it could help exploit approval gates in deployed systems. We consider disclosure net-positive. The vulnerable pattern follows from documented SDK behavior, the fix is cheap and standard, and the canary-based testbed lets maintainers test their own gates. We release no exploit against a real system, and all secrets are canaries. A defense that looks robust could also create false confidence, so we list its known gaps prominently.

Responsible conduct. During development, model-written code briefly ran on a developer machine before grading was containerized. An audit of all 148 executed files found only `json`, `os` and `http.server` usage, with no harmful calls. An early grader also passed the host environment to the server under test; the affected local records were redacted and never published.

7 Future Work

- dependency manifests, so approvals bind every file the plan reads;
- compare-and-swap protection for the file the agent writes;
- structured criteria, and separating the approval *contract* from the human-readable plan;
- a different-family verifier and approver, threshold calibration at scale, and reasoning length as an uncertainty signal;
- stronger injections and subtler payloads, attackers that observe the window, repeated edits (approval fatigue), and many concurrent editors;
- real coding agents with permission prompts, the BashArena environment, more models and larger n;
- packaging the defense as an A2A "approval-binding" extension.

The full list is in `a2a_testbed/FUTURE_WORK.md`.

8 Conclusion

A2A approval gates are vulnerable to a simple TOCTOU race that real LLM agents follow 55–100% of the time. Binding the approval to the exact content and acting on what was checked closes the race. Verifying concurrent changes against the plan's decision criteria, and escalating anything not clearly safe, kept all 60 attacks out without discarding legitimate work. The remaining problems are about precision and scope: better criteria, a stronger approver, and binding every file a decision depends on.

Reproducibility. Code, fixtures, run data with full transcripts, the call ledger, the sandbox self-test and the validator are in the repository (`a2a_testbed/`, `artifacts/a2a_runs/`).

References

[1] A2A Project. *Agent2Agent Protocol Specification*, v1.0.0. `https://a2a-protocol.org/v1.0.0/specification/`

[2] Lotfi, A., Rahman, M. M., Karim, I., & Bertino, E. (2026). *A2ABreak: Systematic Security Analysis of the A2A Protocol*. ACSAC 2026. `https://arxiv.org/abs/2609.10871`

[3] Li, T., Chu, C., Zheng, Y., Zhang, B., Gong, N. Z., & Xiao, C. (2026). *A2ASecBench: A Protocol-Aware Security Benchmark for Agent-to-Agent Multi-Agent Systems*. ICLR 2026. `https://openreview.net/forum?id=LfdFnakqGJ`

[4] Lyu, H., Zhang, D., Wu, M., Wei, X., & Chen, H. (2026). *CoAgent: Concurrency Control for Multi-Agent Systems*. https://arxiv.org/abs/2606.15376

[5] Peng, Y., & Wu, X. (2026). *Stateful Governance for Concurrent Agentic Systems*. https://arxiv.org/abs/2608.02764

[6] Lyu, Y., Ren, Y., Lai, R., & Liu, W. (2026). *From Version Conflicts to Decision Conflicts: Selective Revalidation for Long-Running AI Agents*. https://arxiv.org/abs/2609.08015

[7] Bishop, M., & Dilger, M. (1996). Checking for race conditions in file accesses. *Computing Systems*, 9(2).

[8] Kung, H. T., & Robinson, J. T. (1981). On optimistic methods for concurrency control. *ACM Transactions on Database Systems*, 6(2).

[9] Greshake, K., et al. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *AISec '23*.